Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Инженерный бизнес и менеджмент

КАФЕДРА Промышленная логистика

ЛАБОРАТОРНАЯ РАБОТА №2

Выполнил студент ИБМ3-54Б Сергеева М.А.

(Группа, подпись, дата) (И.О.Фамилия)

Проверил _____ Бурмистрова М.В.

(Подпись, дата) (И.О.Фамилия)

2026 г.

1. Цель работы

Знакомство с инструментами построения пользовательских интерфейсов веб-сайтов: HTML, CSS и JavaScript. В ходе работы продолжена реализация простого калькулятора, вёрстка которого была выполнена в ЛР 1: добавлена программная логика на JavaScript, после чего выполнены задания для самостоятельной проработки.

2. Постановка задачи

1. Подключить JavaScript к HTML-странице.
2. Организовать доступ к HTML-элементам калькулятора из JavaScript.
3. Запрограммировать кнопки калькулятора: цифры, десятичную точку, арифметические операции, «=» и очистку.
4. Запустить калькулятор в браузере с помощью Live Server.
5. Выполнить задания для самостоятельной проработки.

3. Исходные материалы

Работа выполнена на основе следующих файлов проекта:

Файл	Назначение
`calculator.html`	Разметка страницы и скрипт с логикой калькулятора
`style.css`	Оформление в пиксельном стиле сайта Deltarune
`logo.png`	Логотип игры над калькулятором
`bg.gif`	Фоновое изображение (город из игры)

Визуальное оформление (палитра, пиксельный шрифт Press Start 2P, квадратные кнопки с белой окантовкой, акцентный цвет `#09ABD3`) было получено в ЛР 1 и в этой работе не менялось, за исключением небольших добавлений, описанных ниже.

4. Ход работы

4.1. Подключение JavaScript

Существует два способа подключить JavaScript: встроенный скрипт в теге `<script>` и внешний файл `.js`. В методических указаниях рекомендован второй способ. В данной работе использован первый вариант: код размещён в теге `<script>` в конце `<body>`, после всей разметки калькулятора. Такое расположение гарантирует, что к

моменту выполнения скрипта все элементы уже созданы, поэтому обёртка `window.onload` не требуется.

```
<script>
  // ===== Логика калькулятора =====
  const screen = document.getElementById('result');
  ...
  render();
</script>
```

4.2. Разметка калькулятора

Экран калькулятора — элемент `div` с идентификатором `result`. Кнопки расположены в контейнере `.keys` (сетка из четырёх столбцов). Идентификаторы кнопок построены по единой схеме, что позволяет обращаться к ним по имени:

Группа	Идентификаторы
Цифры	`btn_digit_0` … `btn_digit_9`
Десятичная точка	`btn_digit_dot`
Операции	`btn_op_plus`, `btn_op_minus`, `btn_op_mult`, `btn_op_div`
Результат	`btn_op_equal`
Служебные	`btn_op_clear`, `btn_op_sign`, `btn_op_percent`

4.3. Доступ к HTML-элементам из JavaScript

Элементы получены методом `document.getElementById`. Результаты поиска сохранены в константах, чтобы не обращаться к DOM повторно:

```
const screen = document.getElementById('result');

const opButtons = {
  '+': document.getElementById('btn_op_plus'),
  '-': document.getElementById('btn_op_minus'),
  '*': document.getElementById('btn_op_mult'),
  '/': document.getElementById('btn_op_div')
};
```

Объект `opButtons` связывает символ операции с кнопкой. Он нужен для подсветки операции, ожидающей второго операнда.

Кнопки с цифрами обрабатываются в цикле: идентификатор собирается из префикса `btn_digit_` и номера цифры. Это позволяет не перечислять десять кнопок вручную.

```
for (let i = 0; i <= 9; i++) {
  document.getElementById('btn_digit_' + i)
    .addEventListener('click', () => inputDigit(String(i)));
}
```

4.4. Состояние калькулятора

Число вводится по цифрам, поэтому текущее значение хранится как строка. Для хранения состояния использованы четыре переменные:

Переменная	Назначение
`cur`	число на экране (строка)
`prev`	первый операнд
`op`	выбранная операция: `+`, `-`, `*`, `/`
`fresh`	признак того, что следующая цифра начинает новое число

Константа `MAX_DIGITS = 12` ограничивает длину вводимого числа.

4.5. Ввод цифр и десятичной точки

Функция `inputDigit` добавляет цифру в конец строки `cur`. Если установлен флаг `fresh`, ввод начинается с нуля. Ведущий ноль заменяется введённой цифрой, а общая длина числа не превышает `MAX_DIGITS`. Функция `inputDot` добавляет точку, только если в числе её ещё нет, поэтому ввод вида `1.2.3` невозможен.

```
function inputDigit(d) {
  if (isError()) clearAll();
  if (fresh) { cur = '0'; fresh = false; }
  if (cur === '0') cur = d;
  else if (cur.replace(/[-.]/g, '').length < MAX_DIGITS) cur += d;
  render();
}
```

4.6. Арифметические операции и кнопка «=»

Вычисление вынесено в функцию `compute`, где операция выбирается оператором `switch`:

```
function compute(a, b, o) {
  switch (o) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return b === 0 ? NaN : a / b;
  }
  return b;
}
```

Функция `setOp` вызывается при нажатии на кнопку операции. Если операция уже выбрана и введён второй операнд, то перед запоминанием новой операции вычисляется промежуточный результат. Благодаря этому поддерживаются цепочки вида `2 + 3 + 4`. Функция `equals` вычисляет результат, сбрасывает операцию и выводит значение на экран.

Деление на ноль не приводит к бесконечности на экране: `compute` возвращает `NaN`, а функция `format` превращает нечисловой результат в надпись «Ошибка». Ввод любой цифры после ошибки сбрасывает калькулятор.

4.7. Форматирование результата

В JavaScript числа с плавающей точкой дают погрешности, например `0.1 + 0.2 = 0.30000000000000004`. Чтобы на экране не появлялись такие «хвосты», результат округляется до 12 значащих цифр. Если запись всё равно не помещается на экран, число переводится в экспоненциальный вид.

```
function format(n) {
  if (!isFinite(n)) return 'Ошибка';
  let s = String(parseFloat(n.toPrecision(12)));
  if (s.length > MAX_DIGITS + 1) s = n.toExponential(5);
  return s;
}
```

4.8. Вывод на экран

Функция `render` выводит значение `cur` в элемент `result` через свойство `textContent`. Она же выполняет два действия по оформлению:

- для длинных чисел (более девяти символов) добавляет класс `small`, который уменьшает шрифт, чтобы число помещалось в окне;
- добавляет класс `active` кнопке операции, которая ожидает второго операнда. В CSS такая кнопка инвертируется: чёрный фон и акцентный цвет текста.

4.9. Обработчики событий

Для привязки обработчиков использован метод `addEventListener` с анонимными и именованными функциями. Этот способ считается современным стандартом: он позволяет назначить несколько обработчиков на одно событие и допускает их удаление во время работы программы.

```
document.getElementById('btn_op_plus').addEventListener('click', () =>
setOp('+'));
document.getElementById('btn_op_equal').addEventListener('click', equals);
document.getElementById('btn_op_clear').addEventListener('click', clearAll);
```

После каждого клика кнопка теряет фокус (`blur()`). Иначе нажатие клавиши Enter повторно «нажимало» бы последнюю кликнутую кнопку.

Отдельно выделено, что кнопка «Тема» использует inline-обработчик `onclick` прямо в HTML. Для короткого однострочного действия это допустимо, но методические указания не рекомендуют такой подход: он смешивает разметку и логику, а обработчик нельзя заменить или удалить во время выполнения. При дальнейшей доработке его стоит перенести в скрипт.

4.10. Управление с клавиатуры

Дополнительно к заданию реализовано управление с клавиатуры: на весь документ повешен обработчик события `keydown`.

Клавиша	Действие
`0`–`9`	ввод цифры
`.` или `,`	десятичная точка
`+`, `-`, `/`	выбор операции
`*`, `x`, `X`	умножение
`Enter` или `=`	вычисление результата
`Backspace`	стирание последней цифры
`Escape`, `c`, `C`	очистка
`%`	вычисление процента

Сочетания с `Ctrl`, `Alt` и `Meta` игнорируются, чтобы не мешать системным сочетаниям клавиш. Для обработанных клавиш вызывается `preventDefault()`.

4.11. Запуск через Live Server

Для просмотра калькулятора в браузере использовано расширение Live Server для VS Code: после установки расширения в нижней панели нажата кнопка Go Live. Страница открывается по локальному адресу, а изменения в коде применяются в браузере автоматически.

[Скриншот: кнопка Go Live в VS Code и калькулятор в браузере]

5. Задания для самостоятельной проработки

5.1. Статус выполнения

№	Задание	Статус
01	Смена знака +/-	Выполнено
02	Вычисление процента %	Выполнено
03	Кнопка стирания последней цифры (backspace)	Частично: работает по клавише Backspace, отдельной кнопки на экране нет

№	Задание	Статус
04	Смена цвета фона по кнопке	Выполнено (кнопка «Тема»)
05–12	Квадратный корень, квадрат, факториал, «000», накапливаемое сложение и вычитание, смена цвета окна вывода, индивидуальная операция	Не реализованы

5.2. Смена знака (задание 01)

Функция `toggleSign` добавляет к строке минус или убирает его. Работа со строкой, а не умножение на −1, сохраняет введённую дробную часть, например `5.` не превращается в `-5`. Для нуля и для состояния ошибки функция ничего не делает.

```
function toggleSign() {
  if (isError() || cur === '0') return;
  cur = cur.startsWith('-') ? cur.slice(1) : '-' + cur;
  render();
}
```

5.3. Процент (задание 02)

Функция `percent` делит текущее число на 100 и пропускает результат через `format`. После неё устанавливается флаг `fresh`, так что следующая введённая цифра начинает новое число.

```
function percent() {
  if (isError()) return;
  cur = format(parseFloat(cur) / 100);
  fresh = true;
  render();
}
```

5.4. Стирание цифры (задание 03)

Функция `backspace` с помощью метода `slice` удаляет последний символ строки. Если остаётся пустая строка или одиночный минус, на экран возвращается `0`. Сразу после вычисления (когда `fresh` равно `true`) стирание не выполняется, чтобы не портить результат.

```
function backspace() {
  if (isError() || fresh) return;
  cur = cur.length > 1 && cur !== '-0' ? cur.slice(0, -1) : '0';
  if (cur === '-') cur = '0';
  render();
}
```

Функция подключена к клавише Backspace (см. п. 4.10). Для полного выполнения задания остаётся добавить экранную кнопку, например на место одной из служебных клавиш.

5.5. Смена цвета фона (задание 04)

Кнопка «Тема» переключает класс `alt` у элемента `body`:

```
<button type="button" class="my-btn tool"
        onclick="document.body.classList.toggle('alt')">Тема</button>
```

Фон страницы — это изображение `bg.gif`, выведенное через псевдоэлемент `body::before`. Правило для класса `alt` применяет к нему CSS-фильтр поворота оттенка, из-за чего синий фон становится фиолетовым:

```
body.alt::before {
  filter: hue-rotate(60deg);
}
```

Отличие от подсказки к заданию (свойство `style.backgroundColor`) в том, что цвет меняется не прямой записью в стиль, а переключением CSS-класса. Это позволяет оставить всё оформление в таблице стилей, а повторное нажатие возвращает исходный вид.

[Скриншот: калькулятор с обычным фоном и после нажатия «Тема»]

6. Результаты работы

Для проверки работы выполнены следующие сценарии:

Сценарий	Ожидаемый результат
`7 + 8 =`	на экране `15`
`2 + 3 + 4 =`	промежуточный результат `5` после второго плюса, итог `9`
`0.1 + 0.2 =`	`0.3`, без «хвоста» погрешности

Сценарий	Ожидаемый результат
`5 / 0 =`	надпись «Ошибка»
`1.2.3`	вторая точка игнорируется, на экране `1.23`
`50 %`	`0.5`
`9 +/-`	`-9`
ввод более 12 цифр	лишние цифры не добавляются

[Скриншот: пример вычисления с несколькими операциями]

[Скриншот: сообщение «Ошибка» при делении на ноль]

[Скриншот: подсветка кнопки ожидающей операции]

7. Выводы

В ходе работы вёрстка калькулятора из ЛР 1 дополнена логикой на JavaScript. Рассмотрены способы подключения скрипта к странице, получение элементов DOM через `getElementById`, а также способы назначения обработчиков событий и их сравнительные особенности. Реализованы ввод чисел, четыре арифметические операции, цепочки вычислений, очистка, смена знака, процент и управление с клавиатуры. Обработаны граничные случаи: повторная десятичная точка, деление на ноль, погрешности чисел с плавающей точкой и переполнение экрана. Из заданий для самостоятельной проработки выполнены 01, 02 и 04, задание 03 выполнено частично. Остальные задания (05–12) остаются направлением для дальнейшей доработки.